IT Workshop II (Cloud Computing): Hadoop - Concepts

October 5, 2026

Apache Hive: Introduction

- Hive is a data warehouse infrastructure tool to process structured data in Hadoop. It resides on top of Hadoop to summarize Big Data, and makes querying and analyzing easy.
- Initially Hive was developed by Facebook, later the Apache Software Foundation took it up and developed it further as an open source under the name Apache Hive.
- Hive is designed for OLAP.
- It provides SQL type language for querying called HiveQL or HQL.

Apache Hive: Introduction

- It is familiar, fast, scalable, and extensible.
- In normal use, Hive runs on your workstation and converts your SQL query into a series of MapReduce jobs for execution on a Hadoop cluster. Hive organizes data into tables, which provide a means for attaching structure to data stored in HDFS.
- Metadata— such as table schemas—is stored in a database called the metastore.

Hive - Statements

- **Create Database:** CREATE DATABASE [IF NOT EXISTS] userdb;
- **Create Table:**

```
CREATE TABLE IF NOT EXISTS employee ( eid int, name String,
salary String, destination String)
COMMENT 'Employee details'
ROW FORMAT DELIMITED
FIELDS TERMINATED BY '\t'
LINES TERMINATED BY '\n'
STORED AS TEXTFILE;
```

Hive - Statements

- **Load Data:** LOAD DATA LOCAL INPATH '/home/user/sample.txt' OVERWRITE INTO TABLE employee;
- **SELECT** statement is used to retrieve the data from a table. **WHERE** clause works similar to a condition. It filters the data using the condition and gives you a finite result.
  SELECT * FROM employee WHERE salary $>$ 30000;
- **ORDER BY:** The ORDER BY clause is used to retrieve the details based on one column and sort the result set by ascending or descending order.
  SELECT Id, Name, Dept FROM employee ORDER BY DEPT;

Hive - Statements

- **GROUP BY:** The GROUP BY clause is used to group all the records in a result set using a particular collection column.
  SELECT Dept,count(*) FROM employee GROUP BY DEPT;
- **JOIN:** Join is a clause that is used for combining specific fields from two tables by using values common to each one.
  SELECT c.ID, c.NAME, c.AGE, o.AMOUNT
  FROM CUSTOMERS c JOIN ORDERS o
  ON (c.ID = o.CUSTOMER_ID);

Apache Pig - Introduction

- Apache Pig raises the level of abstraction for processing large datasets.
- MapReduce allows us to specify a map function followed by a reduce function. But, working out how to fit data processing into this pattern, which often requires multiple MapReduce stages, can be a challenge.
- With Pig, the data structures are much richer, typically being multivalued and nested. The transformation that we can apply to the data are much more powerful. It includes joins, for example.

Apache Pig - Introduction

- Pig is made up of two pieces:
  - The language used to express data flows, called Pig Latin.
  - The execution environment to run Pig Latin programs. There are currently two environments: local execution in a single JVM and distributed execution on a Hadoop cluster.
- A Pig Latin program is made up of a series of operations, or transformations, that are applied to the input data to produce output.
- Pig is a scripting language for exploring large datasets. Pig is designed to be extensible. Loading, storing, filtering etc can be altered by user-defined functions (UDFs).

Apache Pig - Execution Types

Pig has two execution types or modes: local mode and MapReduce mode.

- **Local mode:** In local mode, Pig runs in a single JVM and accesses the local filesystem. The execution type is set using the -x or -exectype option. To run in local mode, set the option to local:
  % pig -x local
  grunt>
- **MapReduce mode:** In MapReduce mode, Pig translates queries into MapReduce jobs and runs them on a Hadoop cluster.

Apache Pig - An Example

Pig has two execution types or modes: local mode and MapReduce mode.

- Start up Grunt in local mode, then enter the first line of the Pig script:
  records = LOAD '/home/user/sample.txt' AS (year:chararray, temperature:int, quality:int);
- Relations are given names, or aliases, so they can be referred to. This relation is given the records alias. We can examine the contents of an alias using the DUMP operator:
  DUMP records;
- We can see the relation's schema—using the DESCRIBE operator on the relation's alias:
  grunt> DESCRIBE records;
  records: year: chararray,temperature: int,quality: int

Apache Pig - Joining Data

- Let us look at an example of an inner join. Consider the relations A and B:
  grunt> DUMP A;
  grunt> DUMP B;
- We can join the two relations on the numerical (identity) field in each:
  grunt> C = JOIN A BY $0, B BY $1;
  grunt> DUMP C;

Apache Pig - Group

- The GROUP statement groups the data in a single relation. GROUP supports grouping by more than equality of keys: you can use an expression or user-defined function as the group key.
  For example, consider the following relation A: grunt> DUMP A;
  (Joe,cherry)
  (Ali,apple)
  (Joe,banana)
  (Eve,apple)
- Let's group by the number of characters in the second field:
  grunt> B = GROUP A BY SIZE($1);
- grunt> DUMP B;
  (5,(Ali,apple),(Eve,apple))
  (6,(Joe,cherry),(Joe,banana))

Apache Pig - Sorting Data

- Relations are unordered in Pig. There is no guarantee which order the rows will be processed in. In particular, when retrieving the contents of A using DUMP or STORE, the rows may be written in any order. If you want to impose an order on the output, you can use the ORDER operator to sort a relation by one or more fields.
- The following example sorts A by the first field in ascending order and by the second field in descending order:
  grunt> B = ORDER A BY \$0, \$1 DESC;
  grunt> DUMP B;
- Sometimes you have several relations that you would like to combine into one. For this, the UNION statement is used.

The End